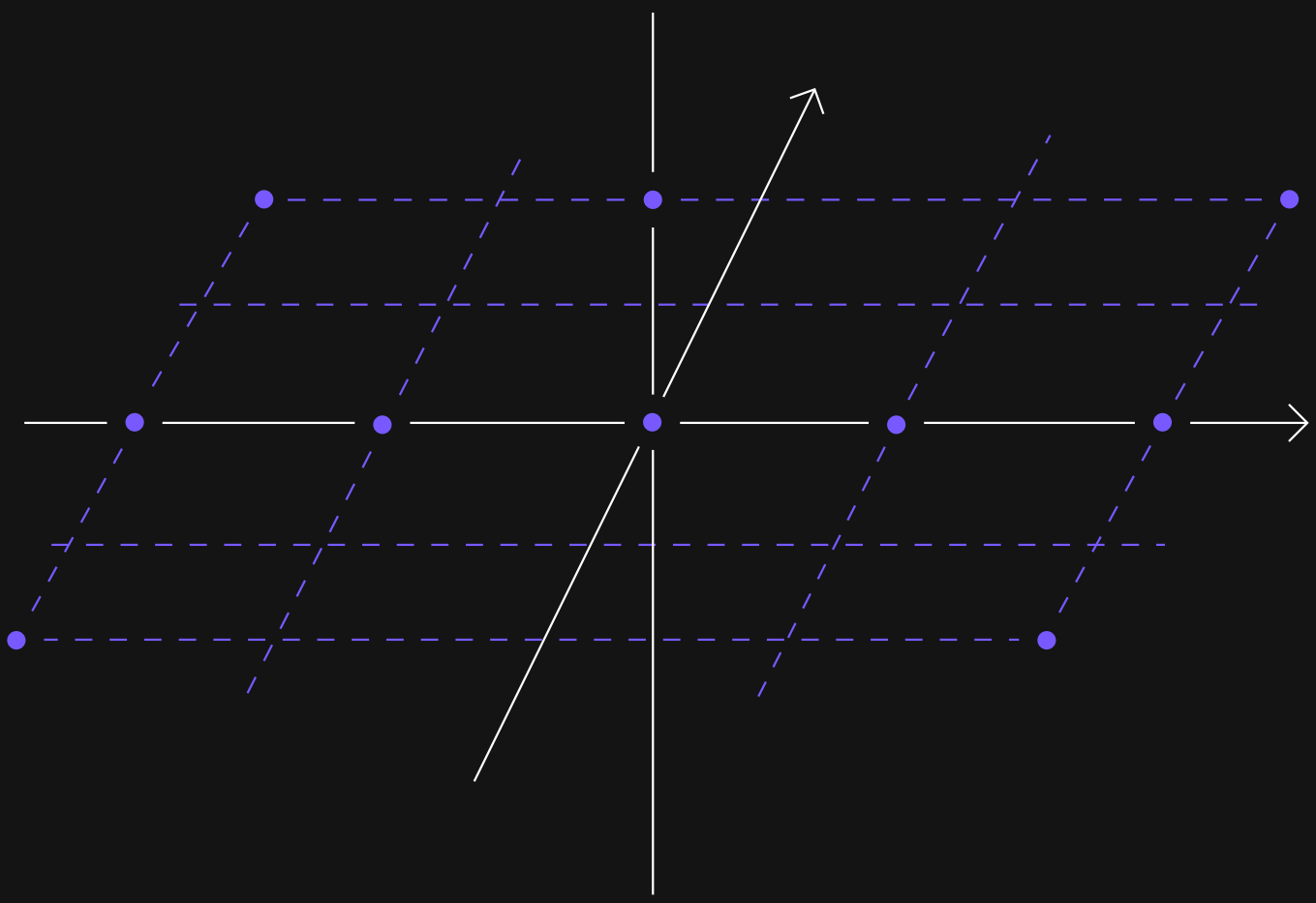
Разработка

Лонгрид

Сетевые технологии

04

QUIC
Потоки, TLS и смена сети



Содержание

1. Задача приложения и место QUIC в стеке

2. Почему UDP и что находится внутри

3. Потоки и пределы независимости

4. Установление соединения и TLS 1.3

5. Возобновление, 0-RTT и повторный эффект

6. Потери, лимиты получателя и перегрузка

7. Connection ID и смена пути

8. HTTP/3 и зависимость QPACK

9. Как клиент обнаруживает HTTP/3 и выбирает другой вариант

10. DNS, датаграммы и туннели: где ещё нужен QUIC

11. Диагностика: что передать коллеге

12. Практические задачи и ответы

13. Краткий справочник и источники

Лекция 4 курса «Сетевые технологии», 2026.

Этот текст продолжает лекции о TLS, TCP и UDP. Начнём с двух знакомых ситуаций: браузер одновременно загружает несколько ресурсов, а телефон переключается между WiFi и мобильной сетью. На их примере разберём потоки QUIC, установление защищённого соединения, потери и смену пути. Затем посмотрим, как эти механизмы используют HTTP/3, DNS и VPN.

Основной рассказ можно читать подряд. Подразделы «Подробнее» объясняют детали, к которым удобно вернуться при разборе трассы. В конце есть задачи с ответами. Числа в расчётах и схемы обменов — учебные модели; результаты реальных измерений зависят от сети и реализации.

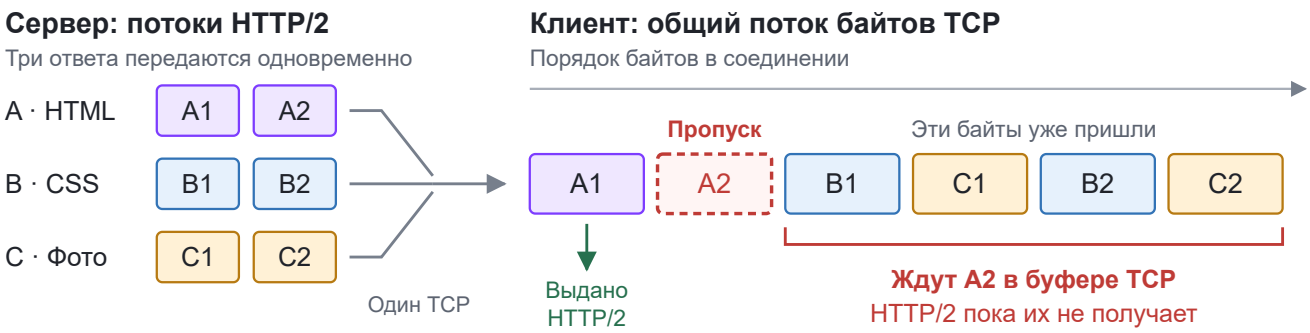
1. Задача приложения и место QUIC в стеке

Несколько обменов в одной сети

Браузер получает страницу, таблицу стилей и изображение. Сервер уже подготовил ответы, но один пакет по дороге потерялся. В другом случае пользователь переключает телефон с Wi-Fi на мобильную сеть посередине загрузки. Для приложения оба случая могут выглядеть как пауза, хотя причины и способы продолжения различны.

В лекции 3 мы объяснили, почему TCP (Transmission Control Protocol) выдаёт приложению последовательность байтов без пропусков. Если поздние байты уже пришли, но перед ними есть недоставленный участок, получатель ждёт восстановления. HTTP/2 позволяет чередовать несколько прикладных обменов внутри одного TCP-соединения. Поэтому пропуск общего TCP-потока может задержать байты, относящиеся к разным HTTP-запросам.

QUIC предоставляет приложению несколько потоков с отдельным порядком байтов, встраивает защищённое установление соединения и позволяет сохранять состояние при некоторых изменениях сетевого пути. Каждый механизм решает свою задачу: потоки позволяют независимо выдавать данные, TLS защищает обмен, миграция помогает пережить смену адреса. Скорость при этом по-прежнему зависит от канала, перегрузки и работы приложения.



A1…C2 — условные части ответов, а не границы TCP-пакетов. TLS на схеме опущен.

HTTP/2 чередует ответы в одном TCP-потоке; пропуск в нём задерживает выдачу последующих байтов.

Карта OSI

OSI (Open Systems Interconnection) разделяет функции взаимодействия на семь уровней. Это учебная карта, а не требование поместить каждый реальный протокол строго в одну строку.

Уровень	Название	Связь с курсом
L7	Прикладной	Лекция 1: HTTP, DNS и другие диалоги
L6	Представления	Лекция 2: представление данных и защита
L5	Сеансовый	Лекция 3: сеанс приложения и транспортное соединение
L4	Транспортный	Лекция 3: TCP/UDP; сегодня: функции QUIC
L3	Сетевой	IP и доставка между узлами
L2	Канальный	Передача в пределах канала
L1	Физический	Передача сигналов

QUIC использует UDP (User Datagram Protocol), но выполняет транспортную работу. То, что библиотека находится в пользовательском процессе, само по себе не делает протокол прикладным по назначению. TLS (Transport Layer Security) тоже пересекает границы учебной классификации: он участвует в установлении защищённого QUIC.

Два стека для знакомого HTTPS-сценария:

```
HTTP/2                          HTTP/3
  │                               │
TLS records                     QUIC
  │                             ├─ TLS 1.3 handshake: ключи и аутентификация
TCP                             └─ защита QUIC-пакетов и транспорт
  │                               │
IP                              UDP
                                  │
                                IP
```

HTTP (Hypertext Transfer Protocol) задаёт смысл запроса и ответа. HTTP/3 задаёт их перенос через QUIC. В дальнейшем используем QUIC v1 как базовую модель. Число в названии HTTP/3 не является номером версии QUIC.

HTTP/3 — один из пользователей QUIC. Транспорт отвечает за доставку и защиту, а прикладной протокол определяет, что означают переданные байты. Поэтому QUIC можно использовать для DNS, файлового доступа и туннелей, не превращая их в HTTP.

Источники: RFC 9000, обзор QUIC, RFC 9001, связь с TLS, RFC 9114, HTTP/3.

2. Почему UDP и что находится внутри

UDP доставляет датаграммы, QUIC организует соединение

Обычный интерфейс UDP позволяет приложению отправлять и получать датаграммы. Он не предоставляет надёжный упорядоченный поток. Эти свойства реализуют конечные точки QUIC: они хранят состояние, подтверждают получение, восстанавливают нужную информацию и ограничивают отправку. Если в системе виден UDP-трафик, из этого нельзя заключить, что приложение работает без надёжности.

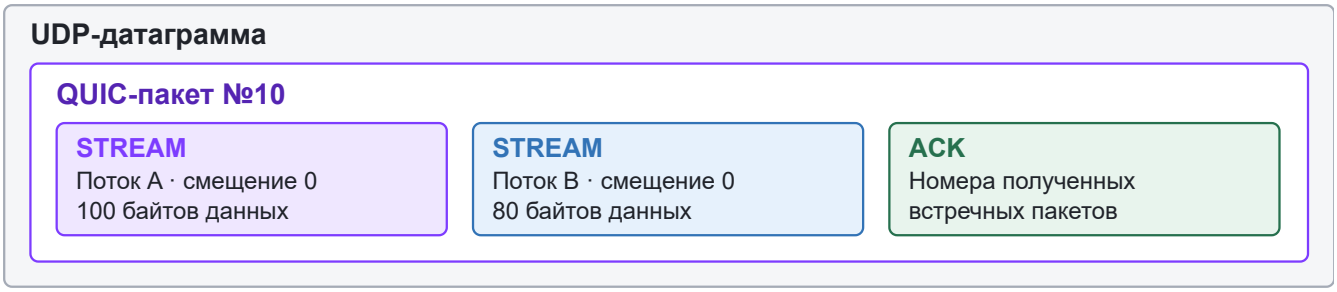
Использование UDP облегчает развёртывание нового транспорта через существующие интерфейсы ОС. Реализацию часто можно обновлять как библиотеку приложения. Однако исполнение только в user space, пользовательском пространстве, не является условием стандарта. Стоимость обработки зависит от реализации и платформы.

Промежуточные устройства тоже имеют значение. NAT (Network Address Translation) изменяет адреса и порты; межсетевой экран может фильтровать UDP или ограничивать время жизни отображения. UDP не гарантирует обход этих ограничений. Успешное TCP-соединение к серверу не доказывает доступность UDP к тому же адресу.

Четыре разные единицы

Единица	Назначение	Чего из неё не следует
Поток, stream	Одна последовательность байтов внутри соединения	Одно чтение не равно сообщению
Фрейм QUIC	Типизированная часть пакета: данные, подтверждение, лимит и др.	Фрейм не обязательно является прикладным сообщением
Пакет QUIC	Заголовок и защищённое содержимое с фреймами	Один пакет не обязательно относится к одному потоку
UDP-датаграмма	Контейнер для передачи через UDP	Одна датаграмма может содержать несколько QUIC-пакетов

Например, пакет несёт STREAM-фрейм с байтами A[0,100), другой STREAM-фрейм с B[0,80) и ACK-фрейм с подтверждением встречных пакетов. Запись [0,100) означает байты со смещениями от 0 до 99, всего 100 байтов. Номер пакета относится к отправке пакета, смещение относится к положению данных внутри потока.



Один пакет может содержать данные нескольких потоков и подтверждения встречных пакетов.

Эта схема показывает вложенность, а не побитовое расположение заголовков. Размеры сообщений приложения, STREAM-фреймов и датаграмм могут различаться.

Подробнее: размер пути

MTU (Maximum Transmission Unit) — максимальный размер пакета соответствующего уровня на участке пути. QUIC должен учитывать ограничения пути и не рассчитывать на IP-фрагментацию как обычный способ передачи больших сообщений. Большой прикладной объект разбивается на данные потоков и затем на пакеты. Минимальная вместимость UDP-полезной нагрузки, требуемая QUIC, — 1200 байтов; это не утверждение, что все QUIC-пакеты всегда имеют такой размер. Отдельные правила увеличения Initial-датаграмм и проверки размера нужны также для безопасности старта.

Источник: RFC 9000, пакеты и фреймы, §§12–14.

3. Потоки и пределы независимости

Порядок внутри потока

Поток QUIC представляет последовательность байтов. Получатель может собрать данные, пришедшие не по порядку, но выдача без пропусков остаётся свойством конкретного потока. Для двух потоков A и B общего порядка выдачи нет.

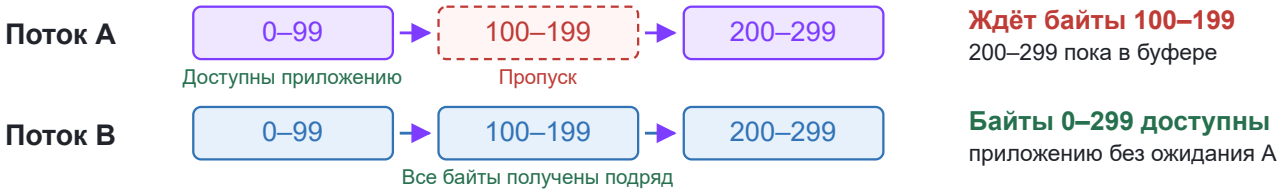
HOL (Head-of-Line blocking) — блокировка из-за ожидания начала последовательности. QUIC устраняет обязательное ожидание потерянных байтов A перед выдачей уже полученных последовательных байтов B. Внутри A ожидание пропуска сохраняется.

Учебная трасса одного направления, все пакеты относятся к уже установленному соединению. До неё данные обоих потоков не приходили.

Событие на получателе	Содержимое	Можно выдать приложению
Пришёл пакет 10	A[0,100)	A[0,100)
Пакет 11 потерялся	A[100,200)	Ничего нового
Пришёл пакет 12	B[0,100)	B[0,100), ожидание A не мешает
Пришёл пакет 13	A[200,300)	Новые байты A пока ждут пропуска
Пришёл пакет 15	A[100,200), восстановление	Можно выдать A[100,300)

Пауза A не означает, что сеть перестала переносить пакеты или B остановился. При этом фраза «потеря затрагивает только один поток» неверна. Если потерянный пакет 11 нёс также B[100,200), пропуск возникнет в обоих потоках.

Одно QUIC-соединение · смещения байтов отдельные для каждого потока



Пропуск внутри A не мешает выдаче уже полученных подряд байтов B.

Общие ресурсы

Потоки делят канал и бюджет управления перегрузкой. Снижение доступного объёма отправки после потери может замедлить будущие данные B. Лимит данных всего соединения тоже общий. А приложение может не использовать ответ B, пока не получит A: например, изображение уже загружено, но его положение задаётся недоставленной таблицей стилей.

Нужно различать три наблюдения: байты пришли на хост, транспорт выдал байты приложению, приложение смогло их использовать. Последнее включает разбор формата и прикладные зависимости. В разделе 8 QPACK даст ещё один конкретный пример.

Подробнее: направления и завершение

Двунаправленный поток имеет два направления передачи, однонаправленный — одно. В двунаправленном потоке смещения считаются отдельно для каждого направления. По идентификатору можно определить инициатора и тип потока, но для диагностики обычно важнее понимать его роль, чем помнить битовое кодирование.

Признак FIN в STREAM-фрейме сообщает конечный размер направления: после получения всех данных до этой границы получатель видит завершение. RESET_STREAM прерывает отправку направления с кодом ошибки и конечным размером. STOP_SENDING просит другую сторону

прекратить отправку. Эти события относятся к потоку. Закрытие QUIC-соединения затрагивает все его потоки.

Надёжность не обещает бесконечно успешную доставку: приложение может отменить поток, соединение может завершиться, путь может исчезнуть. Важны и данные, и сообщение об исходе передачи.

Источник: RFC 9000, streams и состояния, §§2–3.

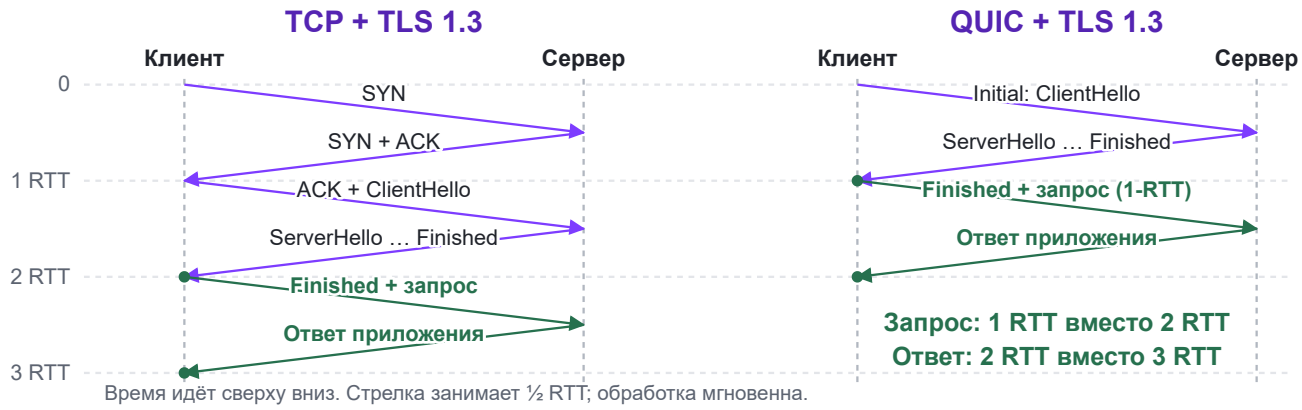
4. Установление соединения и TLS 1.3

Одна согласованная процедура

Клиенту и серверу нужно согласовать ключи, проверить другую сторону по принятой политике и сообщить транспортные параметры. QUIC использует сообщения TLS 1.3 handshake для криптографической части. QUIC сам переносит эти сообщения и защищает свои пакеты. TLS record layer, слой TLS-записей, в этот стек не входит.

CRYPTO-фреймы переносят байты TLS handshake. Они имеют свои смещения для сборки и восстановления, но не являются обычными прикладными потоками. Данные приложения передаются в STREAM-фреймах, а в согласованном расширении — также DATAGRAM.

Возьмём новое соединение с сертификатом сервера. Адрес известен, задержка симметрична, потерь нет, обработку считаем мгновенной. Не используем Retry, HelloRetryRequest, возобновление и TCP Fast Open. RTT (Round-Trip Time) — время прохождения туда и обратно. На диаграмме один проход между сторонами занимает половину RTT. Группы сообщений не означают единственный сетевой пакет.

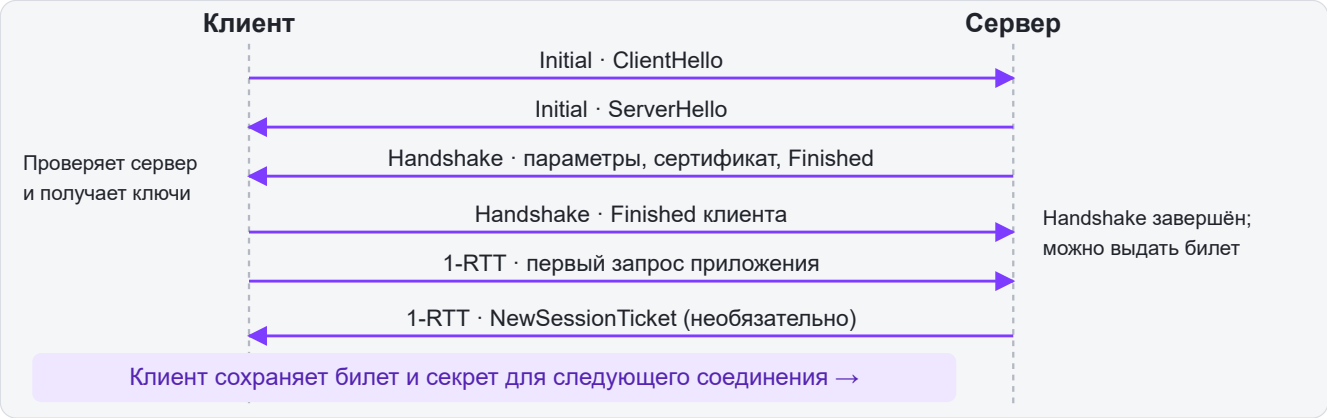


Одинаковые допущения для TCP + TLS и QUIC: запрос через 2 RTT против 1 RTT.

В обычной модели TCP + TLS 1.3 клиент ждёт примерно один RTT для TCP и ещё один для TLS, прежде чем отправить защищённый запрос. С теми же упрощениями ответ приходит примерно через три RTT от начала. QUIC позволяет отправить запрос примерно через один RTT и получить ответ

примерно через два. При RTT 60 мс это 120 мс против 180 мс до ответа в нашей модели, а не обещание ускорить любой сайт на 60 мс. У открытого соединения нового handshake вообще нет.

Сервер может начать отправлять свои 1-RTT-данные раньше, чем получит клиентский Finished. Это не значит, что ответ на ещё не полученный запрос уже существует. Термин 1-RTT обозначает также класс защищённых пакетов и не является таймером для каждого последующего пакета.



Сервер может выдать билет после handshake. Первый запрос не ждёт выдачи билета.

После завершения handshake сервер может прислать `NewSessionTicket` в CRYPTO-фреймах пакета 1-RTT. Клиент сохраняет билет вместе с секретом и параметрами, нужными для следующего подключения. Выдача билета необязательна; приложение уже может обмениваться данными. Рассмотрим использование билета в следующем разделе.

Подробнее: уровни защиты

Уровень	Что обычно несёт	Ограничение интерпретации
Initial	Начало TLS handshake	Ключи выводятся из публично доступного материала; это не конфиденциальность от пассивного наблюдателя
Handshake	Последующие сообщения TLS	Используются ключи стадии handshake
0-RTT	Ранние данные клиента при подходящем сохранённом состоянии	Возможен replay, сервер может отказать
1-RTT	Обычный защищённый обмен	Не означает, что каждый пакет ожидает один RTT

Уровни защиты нельзя путать с пространствами номеров пакетов. Их три: Initial, Handshake и Application Data. Пакеты 0-RTT и 1-RTT используют одно пространство Application Data в соответствующем направлении. Номера не повторяются внутри пространства одного отправителя. У встречного направления свои номера.

TLS проверяет сервер и согласует секреты, но не делает все сетевые метаданные невидимыми. Адреса, UDP-порты, размеры и времена наблюдаемы. Не следует обещать, что любой захват QUIC целиком зашифрован и бесполезен, или, наоборот, что из него можно без ключей прочитать все HTTP-запросы.

Подробнее: дополнительные ожидания и защита адреса

До проверки адреса клиента сервер ограничивает объём отправки относительно полученных данных: базовое ограничение усиления составляет три раза. Оно затрудняет отправку большого объёма жертве, адрес которой подставил атакующий. Большая цепочка сертификатов и недостаток входящих байтов могут заставить сервер ждать. Правила дополнения Initial помогают, но не гарантируют одну волну ответа.

Retry позволяет запросить у клиента подтверждение доступности адреса через токен и обычно добавляет обмен. TLS HelloRetryRequest решает другую задачу: например, серверу требуется другой криптографический параметр. Оба могут добавить ожидание, но проверяют разные условия.

Проверка адреса означает проверку достижимости по пути, а не личности пользователя. Транспортный токен адреса также не равен билету возобновления TLS. После завершения handshake у стороны и после подтверждения handshake в смысле QUIC может пройти разное время; в частности клиент использует HANDSHAKE_DONE от сервера для подтверждения. Это важно для разрешённых действий миграции.

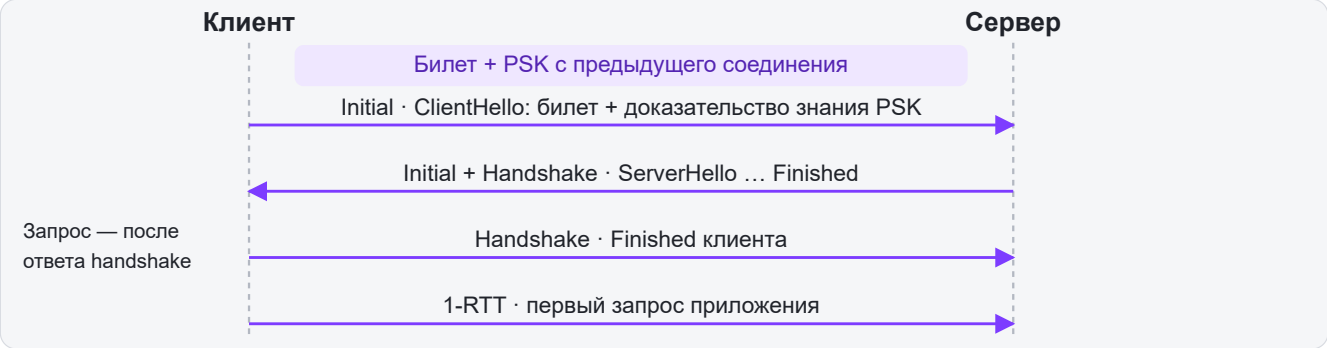
Источники: RFC 9001, использование TLS, RFC 9000, handshake, RFC 9000, проверка адреса.

5. Возобновление, 0-RTT и повторный эффект

Что берём из предыдущего соединения

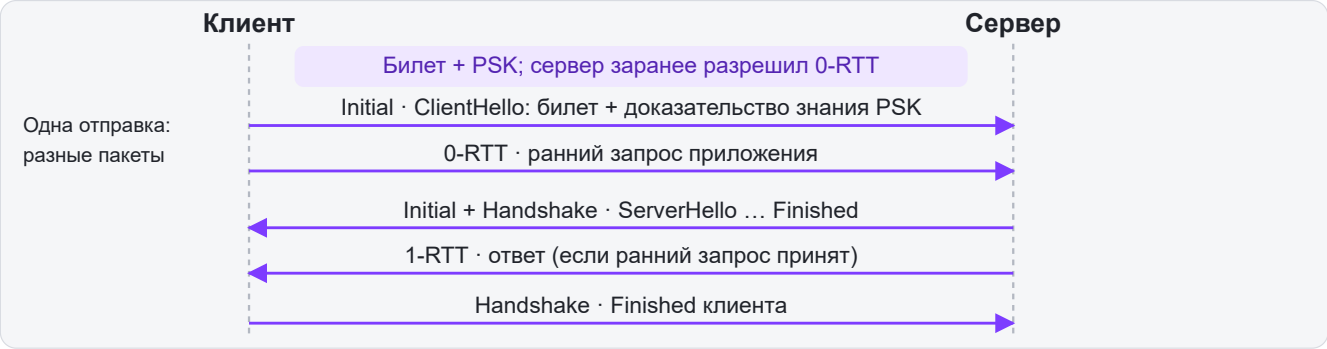
Сервер может выдать NewSessionTicket. Клиент сохраняет билет и связанное состояние, позволяющее позже возобновить TLS. PSK (Pre-Shared Key) означает предварительно согласованный секрет. Билет не следует представлять как отправку этого секрета открытым текстом: стороны получают необходимое криптографическое состояние из предыдущего защищённого обмена.

Возобновление создаёт новое транспортное соединение. Оно может обойтись без повторной передачи сертификата, но не обязано включать ранние данные. Уже открытое соединение — третий случай: оно продолжает работать без нового handshake.



Без 0-RTT запрос отправляется после ответа handshake: билет не устраняет этот RTT.

При 0-RTT клиент использует подходящие ранее согласованные параметры и отправляет ранние данные, не дожидаясь текущего ответа сервера. Они находятся в QUIC 0-RTT-пакетах. Расширение TLS early_data сигнализирует режим, а не служит контейнером HTTP-запроса.



Initial и 0-RTT — отдельные QUIC-пакеты; их можно отправить в одной UDP-датаграмме.

В той же идеализированной модели ответ может прийти примерно через один RTT от начала, если сервер принимает и обрабатывает запрос сразу. «0» относится к ожиданию клиента перед отправкой ранних данных, а не ко времени доставки и не к длительности всей криптографической процедуры.

Replay: повтор может иметь смысл для приложения

Replay — воспроизведение ранних данных в контексте, где сервер может принять их повторно. Атакующему необязательно знать содержимое. Криптографическая целостность пакета и отсутствие обычных дубликатов внутри соединения не дают общей гарантии однократного выполнения ранней операции между соединениями.

Рассмотрим три операции:

Операция	Что меняет повтор	Решение для учебного клиента
Чтение публичного каталога без побочных эффектов	Повторное чтение и дополнительная нагрузка	Кандидат для early data при согласованной политике
Создание заказа	Возможно создание второго заказа	Дождаться handshake; защиту от повторов приложения рассматривать отдельно
GET, который списывает бонус	Повторное списание несмотря на название метода	Недопустим как безопасное раннее чтение

Идемпотентность означает, что повторное применение операции имеет тот же предполагаемый эффект, что однократное. Этого слова недостаточно для оценки реальной системы: остаются побочные эффекты, конкурентные изменения, авторизация и политика серверов. Название GET тоже не доказательство поведения обработчика. Базовый выбор при сомнении — не отправлять операцию в ранних данных.

Отклонение ранних данных

Сервер вправе отклонить 0-RTT, например из-за состояния билета или параметров. Это не обязательно разрывает соединение. Клиент узнаёт об отказе через handshake, а данные могут быть отправлены заново после согласования 1-RTT по правилам прикладного протокола и реализации. Это не следует делать как безусловный повтор любой бизнес-операции при любом тайм-ауте.

У HTTP есть ответ **425 Too Early**: сервер отказывается выполнять запрос в условиях риска ранней обработки. Повтор такого запроса выполняется без early data. HTTP 425 и отказ от 0-RTT на уровне handshake — разные события. Промежуточные HTTP-узлы используют также поле Early-Data, чтобы передать сведения об условиях поступления запроса.

Подробнее: защита имеет границы

Сервер может вести учёт использования билетов и ограничивать принятие ранних данных. Если запросы попадают на несколько независимых экземпляров, локального набора «уже виденных» билетов может быть недостаточно для общей политики. Короткое окно принятия сужает возможность replay, но само по себе не делает его невозможным. Малый размер пакета тоже не решает проблему повторного эффекта.

Ключи 0-RTT выводятся из ранее согласованного секрета; ранние данные не имеют той же гарантии прямой секретности, которую даёт свежий эфемерный обмен для обычного трафика. Это ещё одна причина выбирать ранние операции осознанно.

Источники: RFC 8446, early data, RFC 8446, anti-replay, RFC 9001, 0-RTT в QUIC, RFC 8470, early data в HTTP.

6. Потери, лимиты получателя и перегрузка

Три разные задачи

Получателю не хватает байтов — нужна доставка пропуска. Получателю не хватает места — нужен лимит новых данных. Сеть перегружена — нужно ограничить общий темп отправителя. Во всех случаях приложение может ждать, но наблюдения и следующие проверки различаются.

ACK (Acknowledgment) — подтверждение получения пакетов. В QUIC ACK содержит диапазоны номеров. Например, получены 10 и 12–14, а 11 не подтверждён. Отправитель использует эти сведения и время для оценки потерь. Само отсутствие 11 в одном ACK ещё может объясняться перестановкой пакетов.

Когда нужно восстановить A[100,200), отправитель помещает эти данные в новый пакет, например 15. Данные сохраняют смещение в потоке, пакет получает новый номер. Так можно отличить подтверждение новой отправки от подтверждения старой. ACK относится к приёму пакета транспортом. Он не доказывает, что обработчик HTTP завершил заказ или что пользователь увидел результат.

Подробнее: решение о потере

RFC 9002 использует подтверждения более поздних пакетов и временные критерии. Начальный рекомендуемый packet threshold равен 3: например, подтверждение 14 при неподтверждённом 11 удовлетворяет порогу по разнице номеров при остальных условиях алгоритма. Это не правило «получить три одинаковых ACK». Time threshold помогает оценить потерю по возрасту отправки относительно оценок RTT.

PTO (Probe Timeout) — тайм-аут отправки проб при недостатке подтверждений. Он помогает получить новую информацию и продолжить обмен. Истечение PTO само по себе не объявляет все неподтверждённые пакеты потерянными. Полные формулы таймеров зависят от стадии соединения и задержек подтверждений. При диагностике важно увидеть, какие подтверждения получил отправитель и почему сработал таймер.

Восстанавливается нужная информация, а не буквальная копия любого фрейма: STREAM-данные могут быть отправлены повторно, обновлённый лимит может заменить старый, ACK не нуждается в механическом повторе старого экземпляра. Расширение DATAGRAM не восстанавливает потерянное содержимое.

Flow control: сколько новых данных разрешил получатель

Flow control, управление потоком, защищает ресурсы принимающей стороны. MAX_STREAM_DATA задаёт предел смещения данных потока; MAX_DATA ограничивает суммарный учтённый объём соединения. Это два одновременных ограничения.

Учебный случай без разреженных смещений и отмены потоков:

Ограничение	Разрешено	Учтено	Остаток
Поток A	1000 байтов	900 байтов	100 байтов
Соединение	5000 байтов	4950 байтов	50 байтов

При достаточном бюджете сети можно отправить не более 50 новых байтов A. Увеличение только лимита A результата не изменит. Повтор уже учтённых байтов в новом пакете не расходует квоту новых данных заново, но занимает сеть.

Подробнее: лимит не равен текущему буферу

Лимиты растут и учитывают смещения, поэтому MAX_DATA нельзя трактовать просто как мгновенно свободное место. Приём приложения и выдача нового кредита связаны политикой реализации. Разреженная передача и конечный размер сброшенного потока также влияют на учёт. Отмена не позволяет бесплатно обнулить весь уже объявленный объём и тем самым обойти защиту памяти.

Число одновременно открываемых потоков ограничивается отдельно через MAX_STREAMS. Сообщения STREAM_DATA_BLOCKED и DATA_BLOCKED могут помочь диагностике ожидания, но отсутствие такого события в неполном журнале не доказывает отсутствие ограничения. Нужны также текущие лимиты и состояние отправителя.

Congestion control: сколько выдерживает путь

Congestion control, управление перегрузкой, ограничивает отправку в сеть. cwnd (congestion window) — окно перегрузки, бюджет данных в полёте в выбранной модели. Потоки одного отправителя на пути используют общий бюджет. Поэтому потеря может влиять на будущую скорость всех потоков, даже если выдача уже доставленных байтов B не ждёт A.

RFC 9002 описывает пример контроллера, основанный на NewReno, с начальным ростом, реакцией на перегрузку и восстановлением. Другие допустимые алгоритмы возможны. Pacing распределяет отправку во времени и помогает избегать лишних всплесков; он не является уникальным изобретением QUIC по сравнению со всеми реализациями TCP.

Один и тот же отправитель одновременно ограничен данными приложения, кредитом получателя, окном перегрузки, планированием и возможностями системы. Измеренная скорость ниже пропускной способности не назначает виновника автоматически.

Наблюдение	Рабочая гипотеза	Что сопоставить
Поздние байты A пришли, ранних нет	Пропуск A	Смещения, подтверждения, восстановление
Лимит MAX_DATA достигнут	Не хватает кредита соединения	Учёт данных и обновления лимита
Есть данные и кредит, но бюджет в полёте занят	Ограничение перегрузки	cwnd, подтверждения, RTT, потери
Транспорт доставил всё, запрос ещё не готов	Прикладное ожидание	Состояние декодера и обработчика

Источники: RFC 9002, RFC 9000, flow control, RFC 9000, восстановление информации.

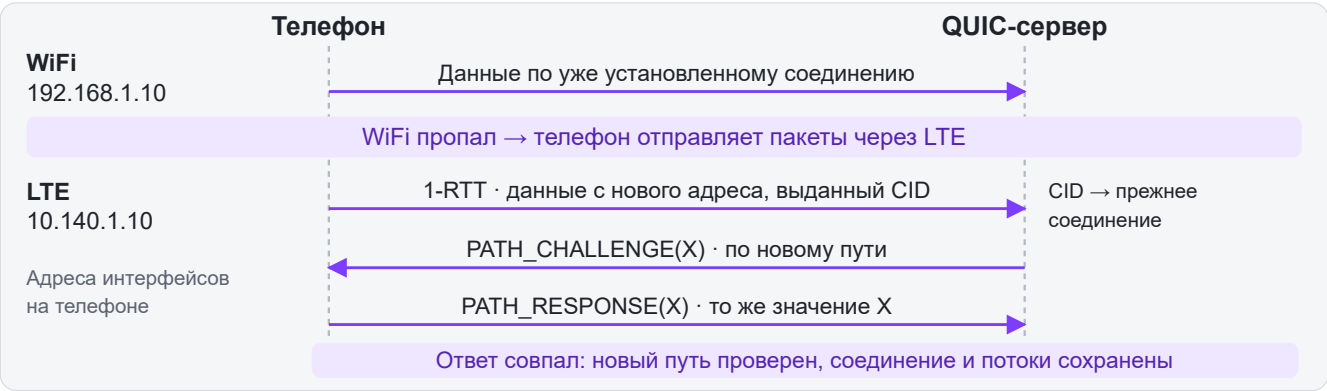
7. Connection ID и смена пути

Состояние соединения и адреса

Пара адрес/порт клиента может измениться при переключении сети или изменении отображения NAT. Connection ID, сокращённо CID, помогает получателю найти состояние QUIC-соединения независимо от прежнего адреса отправителя. IP-адреса и UDP-порты продолжают находиться в пакетах и участвовать в доставке.

CID не является одной вечной строкой: стороны выдают идентификаторы друг другу, используют и выводят их из обращения. Для получения своих пакетов сторона предоставляет идентификаторы другой стороне. Смена CID сама по себе не означает новое соединение. Равным образом один похожий CID в неполном захвате ещё не заменяет проверки состояния и защищённых пакетов.

Учебная схема: handshake уже подтверждён, миграция разрешена, нужные CID имеются.



При смене интерфейса телефон сохраняет соединение, а сервер проверяет новый путь.

Диаграмма показывает проверку нового клиентского адреса сервером; в общем случае стороны могут сами проверять нужные пути. PATH_CHALLENGE — служебный фрейм QUIC с непредсказуемым значением из восьми байт. PATH_RESPONSE возвращает то же значение по пути, откуда пришёл challenge. Эти фреймы обрабатывает транспорт, а не приложение. Проверка показывает достижимость, а криптографическая защита пакетов связывает обмен с установленным состоянием. Знания CID недостаточно для подделки защищённых данных.

В примере телефон получил `192.168.1.10` в WiFi и `10.140.1.10` в LTE. Это частные адреса интерфейсов. Если по дороге есть NAT, сервер видит внешние адреса и UDP-порты WiFi-роутера или оператора. Именно наблюдаемый новый адрес он проверяет. Новый TLS handshake не нужен: стороны уже знают ключи этого соединения. Во время проверки данные могут передаваться с ограничениями; проверка пути не обязательно означает полную остановку приложения.

Ограничения

Активную миграцию инициируют после подтверждения handshake и с учётом транспортных параметров другой стороны. NAT rebinding, изменение отображения NAT без решения приложения мигрировать, нужно отличать от активной смены пути. Параметр disable_active_migration запрещает определённую активную миграцию, но не устраняет необходимость обрабатывать допустимое изменение отображения NAT.

Новый путь может блокировать UDP. Сервер может потерять состояние или завершить соединение по тайм-ауту. Балансировка должна доставить пакет стороне, которая может обработать это состояние. Поэтому наличие CID не гарантирует сохранение любой загрузки при любой смене сети.

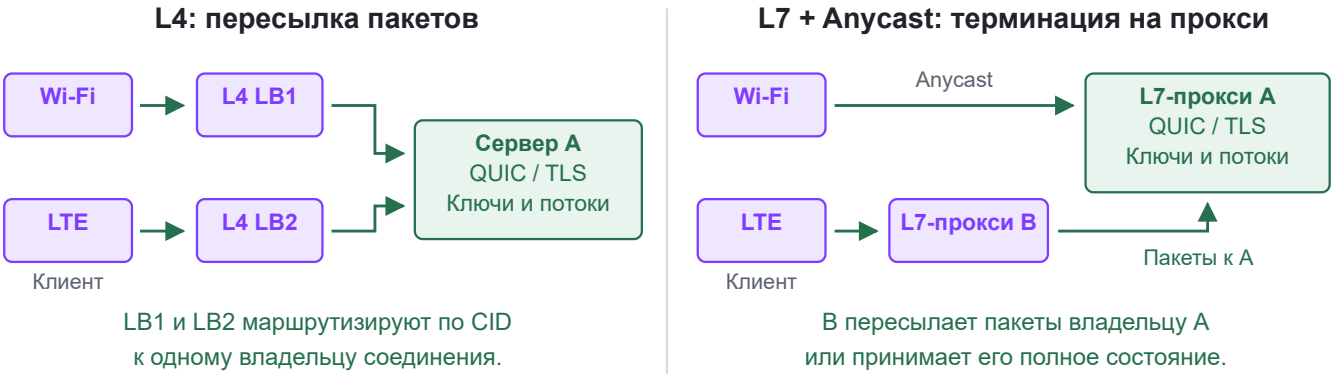
RTT и пропускная способность нового пути могут отличаться. Старый большой бюджет отправки нельзя бездумно перенести на более медленный путь. Базовая модель QUIC v1 описывает миграцию, а не обязательную одновременную передачу по Wi-Fi и мобильной сети.

Где балансировщик хранит соединение

Предположим, по WiFi телефон попадал на вход A, а через LTE — на B. Дальнейшее зависит от того, кто завершает QUIC и хранит ключи, номера пакетов и потоки.

L4-балансировщик пересылает пакеты транспортному серверу. Если оба входа по CID находят одного владельца состояния, смена балансировщика не требует переноса самого соединения. IP и порт клиента могут измениться, а обработчик останется прежним.

L7-прокси сам завершает QUIC/TLS и разбирает HTTP/3. При BGP Anycast сеть может доставить пакет другому прокси, у которого нет нужных ключей и потоков. Чтобы продолжить то же соединение, новый прокси должен переслать пакет прежнему владельцу или получить согласованное полное состояние. Общего TLS-сертификата, билетов возобновления или сессии пользователя в базе для этого недостаточно.



CID помогает найти владельца, но не переносит его ключи, буферы и потоки на другой сервер.

В эксплуатации это ограничивает обещание «QUIC переживает любую смену сети»: нужны доступный путь и доступное состояние на серверной стороне. Если прежний владелец уже завершил соединение, клиенту придётся устанавливать новое.

Подробнее: ротация и закрытие

Ротация CID уменьшает возможность связать активность на разных путях по одному неизменному идентификатору. Однако размеры, времена, другие идентификаторы и возобновление могут оставлять корреляции. Миграция не равна анонимности.

CID нулевой длины допускается в определённых условиях, но ограничивает работу при изменении адреса. Для разбора миграции используем ненулевые CID и журнал их назначения. Сервер может объявить preferred_address, предпочтительный адрес, при handshake; это отдельный предусмотренный случай, а не разрешение любому серверу произвольно менять адрес посреди соединения.

При закрытии CONNECTION_CLOSE сообщает причину. Idle timeout завершает соединение после периода бездействия согласно параметрам и правилам протокола. Stateless reset позволяет узлу, утратившему состояние, сообщить об этом при наличии соответствующего токена. Эти механизмы полезны при диагностике «раньше работало, после паузы перестало», но их формат не входит в базовый опрос.

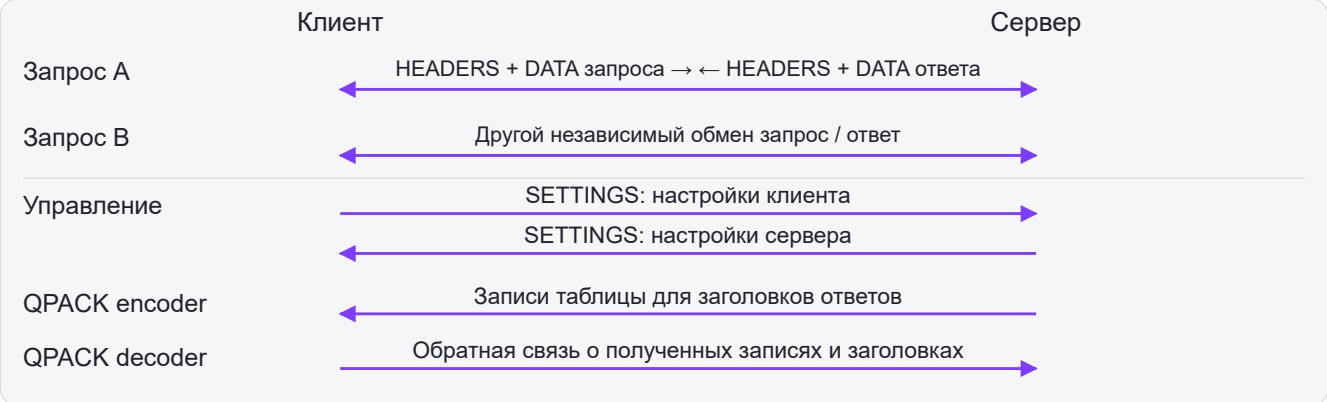
Источник: RFC 9000, CID, миграция и завершение, §§5, 8–10.

8. HTTP/3 и зависимость QPACK

Тот же HTTP-запрос

Приложение по-прежнему запрашивает ресурс, получает статус, поля и тело ответа. HTTP/3 меняет отображение этого обмена на транспорт. Обычный запрос/ответ использует двунаправленный поток QUIC, инициированный клиентом. Служебный обмен использует отдельные однонаправленные потоки.

ALPN (Application-Layer Protocol Negotiation) — согласование прикладного протокола в TLS. Значение h3 означает HTTP/3. Это отличается от выбора версии QUIC, а также от версии TLS. Один номер UDP-порта этих согласований не доказывает.

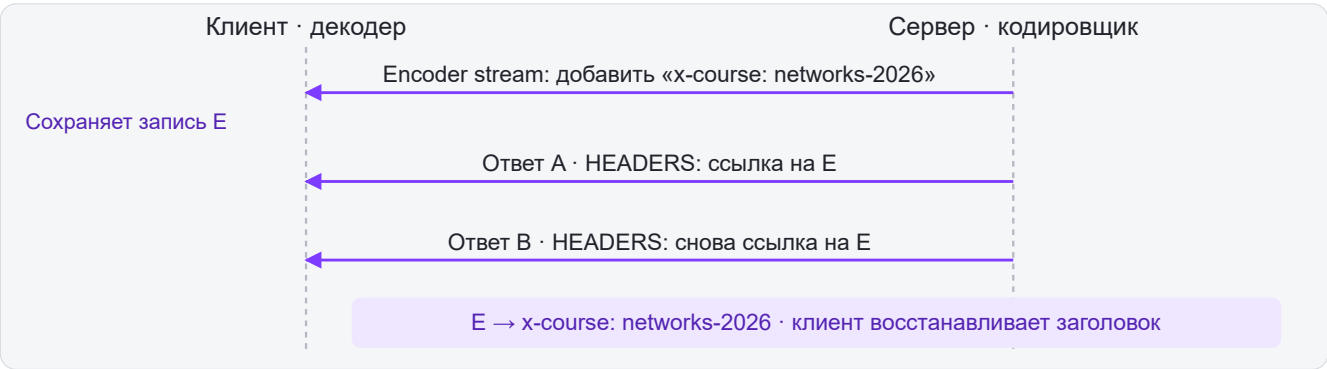


Это карта параллельных потоков одного соединения. Служебные стрелки обозначают отдельные однонаправленные потоки.

HEADERS и DATA здесь являются фреймами HTTP/3. Они составляют байты QUIC-потока, которые транспорт переносит в своих STREAM-фреймах. В захвате пакетов это разные уровни: HTTP/3 разбирает DATA, а QUIC доставляет байты содержащего его потока.

Зачем QPACK

Сервер добавляет к нескольким ответам заголовок `x-course: networks-2026`. Передавать одинаковые имя и значение заново в каждом ответе невыгодно. QPACK позволяет поместить пару в динамическую таблицу, а в блоках HEADERS передавать короткую ссылку на запись. Декодер клиента раскрывает ссылку и восстанавливает исходный заголовок. Здесь назовём запись E; в протоколе используются числовые индексы.

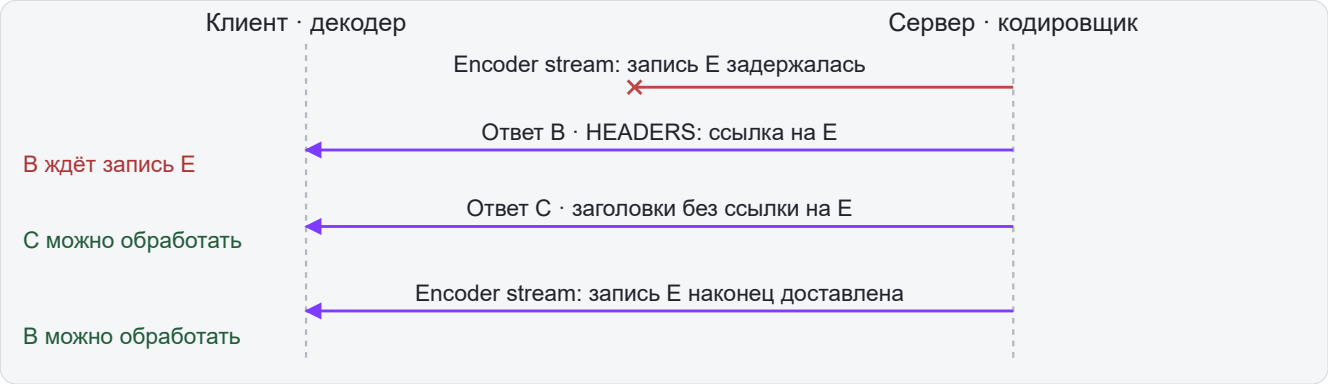


Сначала клиент узнаёт запись E, затем использует её для восстановления заголовков A и B.

Таблица общая для кодирования ответов в этом соединении. Для запросов в обратную сторону используется другое состояние. Обновления передаются по потоку кодировщика, а обратная связь — по потоку декодера. Помимо динамической таблицы есть статическая, заранее известная обеим сторонам, и литералы — значения, передаваемые непосредственно. QPACK сжимает поля HTTP, а сжатие тела, например gzip, решает другую задачу.

Если ссылка пришла раньше записи

Теперь пакет с обновлением таблицы потерялся. QUIC восстановит данные потока кодировщика, но байты ответа B могут прийти раньше: у потоков нет общего порядка. Транспорт уже доставил HEADERS ответа B, однако декодер не может раскрыть ссылку на E. Поэтому обработка B ждёт таблицу.



Когда обновление таблицы доставлено, декодер продолжает обработку B.

Ответ C без такой зависимости можно разобрать сразу. Это и есть граница независимости: QUIC не заставляет потоки ждать общую последовательность байтов, но прикладной формат может создать зависимость между ними. Чтобы избежать такого ожидания, кодировщик использует уже подтверждённые записи, статическую таблицу или передаёт значение целиком. Иногда за меньшую задержку приходится платить несколькими дополнительными байтами.

Подробнее: как ограничивается ожидание

Блок полей сообщает Required Insert Count — сколько вставок требуется для декодирования. Стороны ограничивают вместимость динамической таблицы и число потоков, которые могут блокироваться. SETTINGS_QPACK_BLOCKED_STREAMS — параметр допустимого числа таких потоков. Это ограничение риска, а не обещание отсутствия задержек вообще.

Кодировщик может выбирать ссылки только на подтверждённые записи, статическую таблицу или литералы. Это уменьшает риск ожидания ценой потенциально большего объёма полей. Обратная связь нужна также для безопасного управления временем жизни записей. Полный алгоритм вытеснения таблицы находится в стандарте.

Независимость потоков не устраняет зависимости загрузки страницы: HTML может сообщить о ресурсе только после получения своей части, а обработчик может последовательно обращаться к внутренним сервисам. QUIC не меняет эту логику приложения автоматически.

Источники: RFC 9114, запросы и потоки, RFC 9204, QPACK.

9. Как клиент обнаруживает HTTP/3 и выбирает другой вариант

Объявление, попытка, успех

Клиент должен узнать, что для нужного сервиса существует вариант HTTP/3. Информация может прийти из HTTP-ответа, DNS или сохранённого состояния. Затем клиент пробует соединение и проверяет сервер. У этих этапов разные свидетельства, поэтому фраза «сайт поддерживает h3» не описывает результат конкретного запроса.

Пример поля HTTP-ответа:

```
Alt-Svc: h3=":443"; ma=3600
```

Alt-Svc означает Alternative Services, альтернативные сервисы. Здесь h3 объявляет HTTP/3, пустое имя перед двоеточием сохраняет имя текущего хоста, 443 задаёт порт альтернативного сервиса, ma задаёт срок актуальности объявления в секундах. Это не HTTP redirect: исходный origin, идентичность ресурса, не меняется. Доступность альтернативного адреса и его право обслуживать исходное имя нужно проверить.

Пример DNS-записи:

```
example.test. 3600 IN HTTPS 1 . alpn="h3,h2"
```

Поле	Смысл в примере
example.test.	Имя, для которого опубликована запись
3600	Время хранения DNS-данных в кеше, секунды
IN HTTPS	Класс Internet и тип записи HTTPS
1	Приоритет записи в ServiceMode
.	Целевое имя соответствует имени владельца записи
alpn="h3,h2"	Объявленные прикладные протоколы

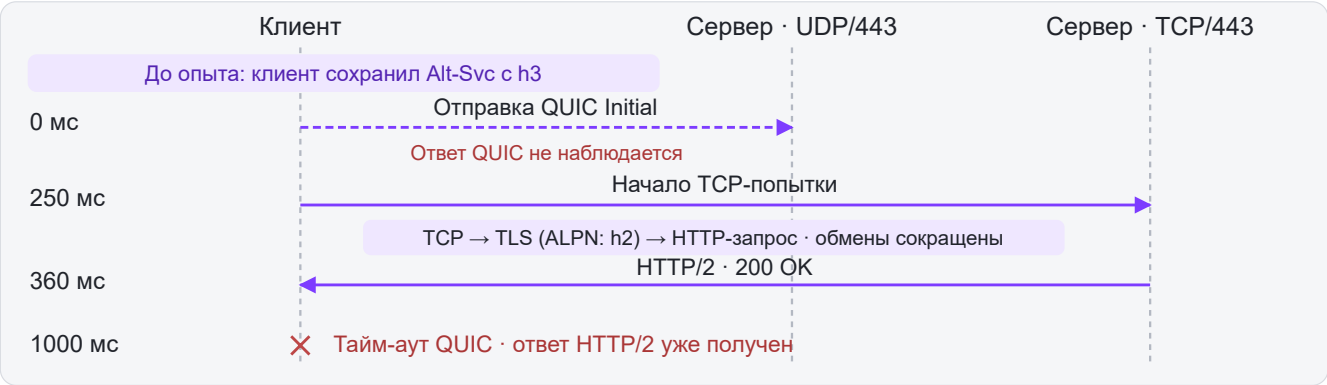
HTTPS здесь — тип DNS-записи, не запрос страницы по HTTPS. Получение записи позволяет узнать параметры до HTTP-обмена. При этом адреса, проверка сервера и особенности клиента остаются частью установления. Пример не утверждает, что любой клиент обязан выбрать h3 или запросить именно этот тип записи.

Fallback

Fallback — использование другого доступного варианта после неудачи или при выборе между попытками. Например, клиент пробует QUIC и параллельно или позднее устанавливает TCP + TLS с HTTP/2. Конкретные задержки, кеширование ошибок и порядок зависят от клиента. Нельзя задавать один универсальный таймер HTTP/3.

Учебные события одного клиента, не реальный захват:

Время	Наблюдение
До опыта	Клиент сохранил Alt-Svc с h3
0 мс	Началась попытка QUIC к объявленному адресу
250 мс	Началась TCP-попытка к тому же сервису
320 мс	Клиент согласовал h2
360 мс	По HTTP/2 пришёл ответ 200
1000 мс	Клиент отметил тайм-аут QUIC; ответных пакетов в его журнале нет



Метки времени соответствуют таблице; расстояния между стрелками не задают масштаб времени.

Доказаны объявление, попытка QUIC и успешный HTTP/2. Блокировка UDP пока лишь гипотеза. Сервер мог быть недоступен по объявленному порту, ответный путь мог терять пакеты, запись могла устареть. Нужно сопоставить наблюдения сервера, адреса и условия другой контролируемой попытки.

Подробнее: что контролировать при сравнении

Проверить одинаковость имени, адреса сервиса, клиента и сети; записать наличие билетов, DNS-кеша, Alt-Svc и уже открытых соединений. Холодный HTTP/2 и повторный HTTP/3 отвечают на разные вопросы. Принудительный режим «только HTTP/3» полезен для изоляции попытки, но не воспроизводит обычный fallback браузера.

Сравнивать несколько повторов и распределение задержек, учитывать неудачные попытки. Если наблюдать только успешные соединения, цена неудачного обнаружения исчезнет из результата. Нужны время до первого полезного ответа, общая длительность, фаза ожидания и условия опыта, а не только имя протокола.

Источники: RFC 7838, Alt-Svc, RFC 9460, HTTPS records, RFC 9114, обнаружение и установление.

10. DNS, датаграммы и туннели: где ещё нужен QUIC

DNS можно передавать разными способами

Задача DNS-клиента остаётся прежней: узнать адрес или другие сведения по имени. Меняется канал до рекурсивного резолвера. Обычный DNS использует UDP или TCP на порту 53. Защищённые варианты добавляют шифрование и проверку сервера.

Вариант	Стек и стандартный порт	Где настраивают
DNS	DNS → UDP или TCP, 53	ОС, роутер, приложение; без шифрования
DoT — DNS over TLS	DNS → TLS → TCP, 853	Системный резолвер или DNS-прокси
DoH — DNS over HTTPS, HTTP/2	DNS → HTTP/2 → TLS → TCP, 443	Браузер, ОС или DNS-прокси
DoH через HTTP/3	DNS → HTTP/3 → QUIC → UDP, 443	Клиент и резолвер с поддержкой этого варианта
DoQ — DNS over QUIC	DNS → QUIC → UDP, 853	Специализированный DNS-клиент или прокси

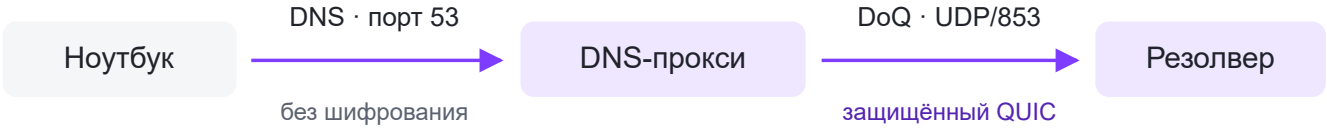
В DoQ нет HTTP: каждый запрос занимает отдельный двунаправленный поток QUIC. Клиент может отправить запросы `alpha.example` и `beta.example`, а резолвер — ответить на второй раньше первого. В DoH через HTTP/3 запросы оформлены как HTTP. Таким образом, DoQ и DoH — разные протоколы, а не вложенные разновидности.

Кто выбирает защищённый DNS

Выбор делает DNS-клиент, а не сайт, который пользователь собирается открыть. Если в браузере настроен DoH, уже первый запрос адреса сайта может идти через HTTPS к резолверу. Настройка не означает «сначала DNS/53, потом QUIC для картинок». При этом самому клиенту нужно знать адрес резолвера: его берут из конфигурации, кеша или получают отдельным начальным разрешением имени — bootstrap.

У Chrome и Firefox есть настройки защищённого DNS с DoH. Конкретный HTTP-транспорт зависит от реализации и резолвера; нельзя считать любой DoH соединением QUIC. Android Private DNS традиционно использует DoT, а для части провайдеров и версий доступен DoH через HTTP/3. В Windows 11 есть системная настройка DoH; в iOS/macOS защищённый DNS можно задавать профилем или приложением. Эти возможности нужно проверять для используемой версии, а не переносить настройки одного устройства на всю аудиторию.

Практический вариант для DoQ — локальный DNS-прокси, например AdGuard Home, с upstream вида `quic://dns.adguard-dns.com`. Ноутбук обращается к прокси обычным DNS, а прокси сам устанавливает QUIC-соединение с резолвером.



Шифрование второго участка не защищает автоматически первый.

DNS внутри VPN тоже защищён только до VPN-шлюза; дальше шлюз может отправить обычный DNS/53. DNSCrypt применяет собственную защиту DNS, а ODoH добавляет HTTPS-реле, чтобы разделить знание адреса клиента и содержимого запроса между участниками. Для ODoH нужны совместимые клиент, реле и резолвер. Во всех случаях спрашивайте: кто видит запрос и какой участок действительно зашифрован?

Свойства DoQ определены в RFC 9250, DoT — в RFC 7858, DoH — в RFC 8484, ODoH — в RFC 9230. Примеры настроек: Firefox, AdGuard Home, DoH3 в Android, DNS в Windows, DNS Settings в Apple.

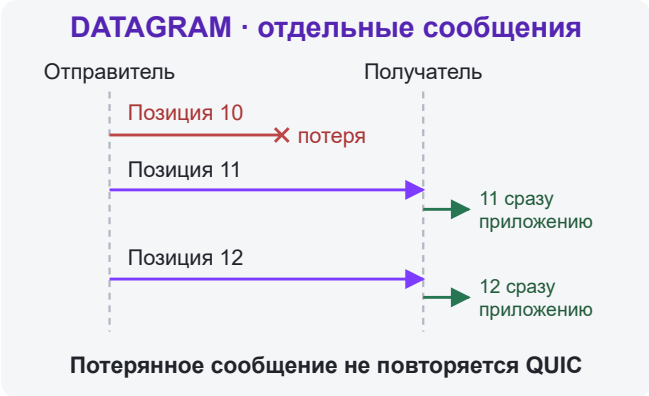
Когда сообщение лучше потерять, чем задержать

Представим игру, которая каждые 50 мс отправляет полное положение игрока. Сообщение 10 потерялось, но уже пришли 11 и 12. Если каждое из них содержит самостоятельное актуальное состояние, восстановление старой позиции может быть менее полезно, чем немедленная обработка новой.

Расширение QUIC DATAGRAM позволяет передавать отдельные сообщения без восстановления потерянного содержимого транспортом. Получатель может сразу передать сообщение 11 приложению. Поток STREAM, напротив, выдаст байты за пропуском только после восстановления

недостающих. Поэтому команды покупки или текст файла разумно отправлять надёжно, а некоторые обновления состояния — датаграммами.





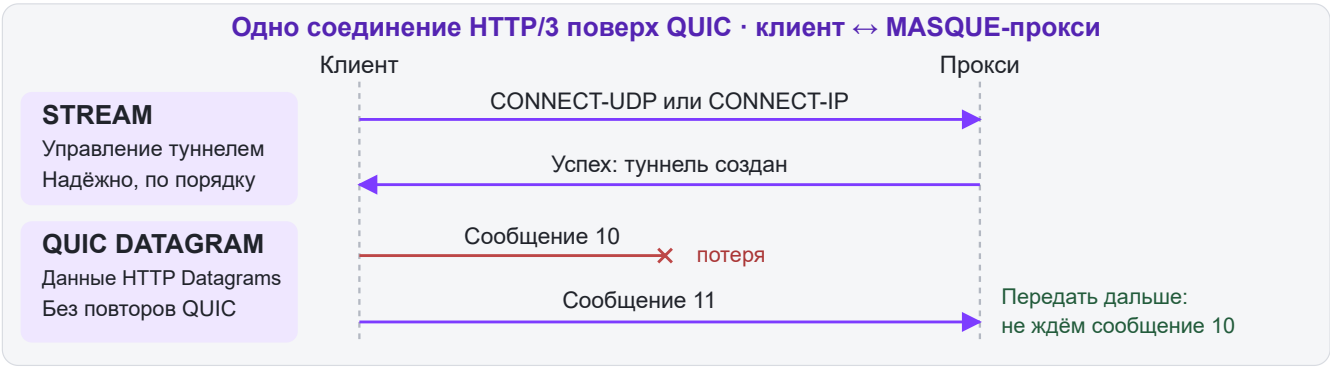
Выбор зависит от смысла данных. Если обновление содержит только разницу с предыдущим, приложению нужен способ восстановить состояние.

DATAGRAM сохраняет шифрование и управление перегрузкой QUIC. Пакет с ним может быть подтверждён ACK, но QUIC не повторяет потерянное сообщение. Поддержку согласуют заранее; размер ограничен, а управление очередью приложения остаётся за разработчиком. Это расширение описано в RFC 9221.

MASQUE: HTTP договаривается о туннеле, QUIC переносит данные

Прокси должен знать, куда пересылать трафик клиента. MASQUE — семейство расширений HTTP для такой договорённости. CONNECT-UDP создаёт проксирование UDP к хосту и порту, CONNECT-IP — туннель для IP-пакетов. QUIC остаётся транспортом под HTTP/3.

В варианте HTTP/3 с QUIC DATAGRAM клиент создаёт туннель надёжным HTTP-запросом в потоке. Датаграммы полезного трафика ассоциированы с этим запросом, но передаются отдельно, а не как байты его STREAM. Поэтому потерянная внешняя датаграмма не заставляет QUIC восстанавливать её перед доставкой следующей.

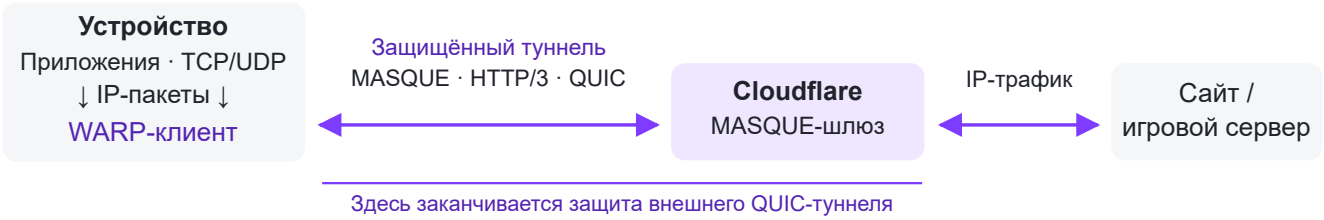


Надёжность управления туннелем не требует превращать весь UDP-трафик в надёжный поток.

Если данные туннеля идут в Capsules по надёжному потоку, ожидание восстановления возвращается. В HTTP/2 поверх TCP оно связано с общей последовательностью TCP. Кроме того, внутреннее приложение само может использовать TCP и ждать его потерь, даже когда внешний

туннель переносит IP-пакеты ненадёжными датаграммами.

Cloudflare WARP — практический пример клиента, который направляет IP-трафик устройства через MASQUE-шлюз. Обычным приложениям и конечным сайтам не требуется поддержка MASQUE: WARP забирает пакеты на устройстве, шлюз выпускает их дальше. Защита внешнего туннеля заканчивается на шлюзе. HTTPS приложения отдельно защищает связь с сайтом. Режим WARP DNS only защищает DNS, а не весь трафик устройства.



Внешний туннель и соединение приложения имеют разные конечные точки.

Источники: CONNECT-UDP, RFC 9298, HTTP Datagrams, RFC 9297, CONNECT-IP, RFC 9484, режимы WARP.

Протоколы приложений поверх QUIC

HTTP/3 не исчерпывает применения транспорта. SMB over QUIC переносит файловый доступ SMB через защищённое QUIC-соединение; практическая реализация есть в поддерживаемых выпусках Windows. MQTT over QUIC применяют для сообщений и телеметрии, например в EMQX. WebTransport даёт веб-приложениям потоки и датаграммы, а Media over QUIC исследует доставку медиаданных. Поддержка и зрелость этих вариантов различаются, поэтому перед выбором сверяют версии клиента и сервера.

Общее у них — возможность использовать потоки, защиту и другие свойства QUIC. Смысл файлов, сообщений, видео или туннелируемых пакетов задаётся протоколом сверху. Собственный VPN поверх QUIC тоже возможен; он не становится MASQUE без реализации соответствующих расширений HTTP.

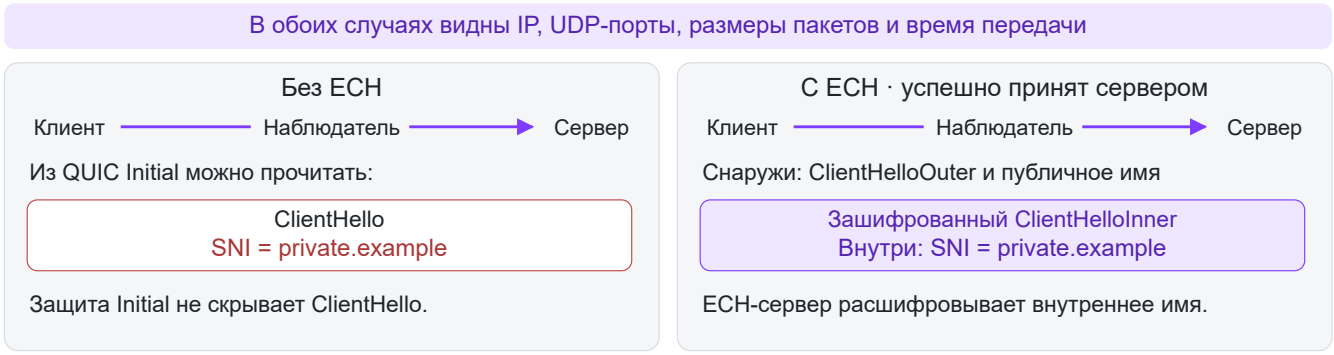
Источники: SMB over QUIC, MQTT over QUIC в EMQX, WebTransport, Media over QUIC.

11. Диагностика: что передать коллеге

Что скрыто от наблюдателя, а что остаётся видимым

QUIC защищает данные приложения, но IP-адреса, UDP-порты, размеры и время пакетов остаются доступны наблюдателю. Даже у Initial есть криптографическая защита, однако её ключи выводятся из публичного материала. Поэтому из Initial можно восстановить ClientHello, в том числе SNI — имя сервера.

ECH (Encrypted Client Hello) — отдельный механизм TLS. Клиент шифрует внутренний ClientHello с настоящим именем для ECH-сервера. Снаружи остаются ClientHelloOuter и публичное имя. Механизм применим и к TLS поверх TCP: наличие QUIC само по себе не означает использование ECH.



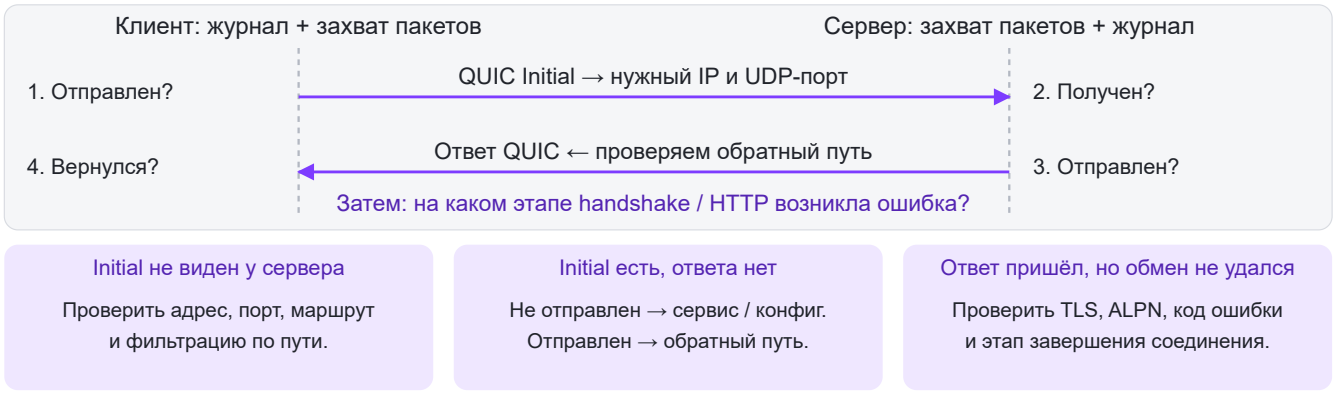
ECH защищает внутренний ClientHello, но не скрывает внешний адрес сервера.

Если DNS-запрос ушёл открытым текстом, наблюдатель всё ещё может узнать нужный домен. Выделенный IP тоже позволяет предположить назначение. Поэтому ECH, шифрование DNS и туннель решают связанные, но разные задачи. Из этого следует и диагностический вывод: фильтрация UDP, ECH и IP — разные гипотезы. Переход на HTTP/2 может обойти проблему UDP, но не обязан помочь с двумя другими.

Источник: TLS ECH, RFC 9849.

Где остановился обмен

Вернёмся к примеру fallback. Клиент сообщил о тайм-ауте QUIC, хотя HTTP/2 получил ответ. Сначала проверим, отправил ли клиент Initial по нужному адресу и порту. Затем найдём ту же попытку в захвате на сервере. Если пакет пришёл, выясним, отправил ли сервер ответ и увидел ли его клиент.



Один и тот же симптом «нет ответа» требует разных проверок в зависимости от наблюдений.

Если Initial не виден на сервере, проверяют адрес назначения, маршрут и фильтрацию, а также правильность захвата. Если Initial есть, но сервер не отвечает, смотрят локальный firewall, слушающий сокет и журнал QUIC. Если ответ отправлен, но клиент его не получил, исследуют

обратный путь и NAT. Наконец, полученный ответ может содержать отказ: тогда важны этап TLS, согласованный ALPN и точная ошибка.

От симптома к следующей проверке

Полезный отчёт позволяет другому инженеру продолжить проверку. В нём нужны время, цель подключения, достигнутая фаза и наблюдаемая ошибка. Фраза «QUIC тормозит» не сообщает, задержалась ли сеть, handshake или обработка запроса.

Источник	Что помогает увидеть	Ограничение
Клиентское приложение	Выбранный протокол, ошибка, фаза и длительность	Не знает всё о сервере и промежуточном пути
Захват пакетов	Адреса, направления, размеры, времена, часть метаданных	Обычный зашифрованный трафик не раскрывает все фреймы и запросы
Журнал QUIC конечной точки	Потоки, ключевые события, лимиты, recovery	Доступность полей и формат зависят от реализации
Серверный журнал	Приход запроса, обработка, ответ	Отсутствие записи требует проверки точки журналирования

В лаборатории можно использовать ключевой журнал своей конечной точки для разбора зашифрованного обмена. Это чувствительный материал: он позволяет читать соответствующий трафик, поэтому его не включают в публичный отчёт с реальными пользовательскими данными. Учебный пакет должен содержать только искусственные запросы. QUIC Initial обычно анализируется без такого журнала, но это не распространяется автоматически на весь обмен.

Шаблон краткого отчёта:

```
Когда и где: дата, часовой пояс, ОС, версия клиента, сеть
Цель: имя сервиса, выбранные адрес и порт
Обнаружение: источник h3, состояние кеша и билета
Попытки: QUIC/TCP, их начало и исход, согласованный ALPN
Симптом: фаза и измеренная длительность
Факты: события с отметками времени и точками наблюдения
Гипотеза: какое объяснение согласуется с фактами
Следующая проверка: какое наблюдение различит оставшиеся причины
```

Пример: «В учебной трассе клиент сохранил Alt-Svc, отправил Initial, не увидел ответных QUIC-пакетов и получил 200 по HTTP/2. Проверим, приходили ли эти датаграммы на сервер и совпадает ли объявленный UDP-порт с доступным сервисом». Это точнее, чем «HTTP/3 на сервере отсутствует».

Подробнее: ограничения измерений

Синхронизация часов влияет на сравнение клиентского и серверного журналов. Разные точки захвата могут видеть разные потери. Пакет, которого нет в журнале приложения, мог дойти до интерфейса и быть отброшен позже. Автоматическая метка анализатора о повторной передаче тоже требует проверки контекста.

Нельзя определить причину потери только по изменению cwnd: это реакция контроллера, а не доказательство конкретного неисправного устройства. Аналогично из одного UDP/443 нельзя уверенно установить прикладной протокол. Сначала сопоставляют уровни наблюдения, затем выбирают эксперимент.

12. Практические задачи и ответы

Задачи

1. В трассе раздела 3 после пакета 13 какие байты доступны приложению? Что изменится, если 11 содержал также B[100,200), а после 13 пришёл пакет 14 с B[200,300)?
2. При RTT 40 мс оцените время отправки клиентского запроса и прихода ответа для нового TCP + TLS 1.3, нового QUIC и принятого QUIC 0-RTT. Используйте допущения раздела 4 и назовите два обстоятельства, меняющих ответ.
3. Повторный PUT устанавливает баланс в заданное значение. Достаточно ли идемпотентности метода, чтобы разрешить early data? Что требуется узнать?
4. В числовой задаче раздела 6 лимит соединения увеличили до 5100. Сколько новых байтов A теперь допускает flow control?
5. Отправитель подтвердил пакеты 10, 12, 13, 14 и восстановил байты потерянного 11 в 15. Что подтверждает последующий ACK для 15?
6. При смене сети изменились и адрес, и CID. Доказывает ли это новое соединение? Какие сведения помогли бы?
7. Все байты блока HTTP-полей пришли, но ссылка на динамическую запись QPACK ещё не разрешена. На каком уровне возникает ожидание?
8. В событиях раздела 9 поддержка h3 объявлена, но ответ получен по h2. Сформулируйте доказанный факт, две гипотезы и одну следующую проверку.
9. Сравните DoQ и DNS через HTTP/3: где находится HTTP? Какое свойство защищённого соединения нельзя автоматически приписать всей цепочке DNS?

Ориентиры для самопроверки

1. В обоих вариантах доступны A[0,100) и B[0,100). В изменённом варианте B[200,300) ждёт восстановления B[100,200), как и более поздние данные A. Потеря одного пакета может создать пропуски в двух потоках.

2. Отправка запроса: 80, 40 и 0 мс от начала соответственно. Ответ: 120, 80 и 40 мс. Это расчётная модель. Потери, обработка, Retry, отказ от ранних данных, неизвестный адрес и повторное использование соединения меняют сравнение.
3. Нет. Узнать фактические эффекты, конкурентные изменения, авторизацию и политику replay. Повтор может перезаписать более новое значение. Для учебного клиента такую операцию откладываем до обычного защищённого обмена.
4. Не более 100 новых байтов: остаток A равен 100, остаток соединения 150. Перегрузка и другие ограничения могут уменьшить фактическую отправку.
5. Получение пакета 15 другой конечной точкой. Это не доказательство выполнения бизнес-операции и не подтверждение получения старого пакета 11.
6. Нет. Нужны назначение и ротация CID, состояние конечной точки и события проверки пути. Даже прежний CID в одиночку не объясняет весь обмен.
7. Декодирование полей HTTP/3, зависимость QPACK. Транспортные байты этого потока уже доставлены; нужна другая последовательность с обновлением таблицы.
8. Факт: попытка QUIC не дала наблюдаемого ответа, HTTP/2 завершился успешно. Гипотезы: фильтрация пути или недоступный объявленный UDP-сервис. Проверка: сопоставить приход и отправку на сервере с клиентскими событиями.
9. В DoQ HTTP отсутствует, в другом варианте DNS использует HTTP/3. Защита связи с резолвером не доказывает защищённость всех его дальнейших запросов и не подтверждает достоверность данных сама по себе.

В своих ответах отделяйте факт от гипотезы. Если данных недостаточно, укажите, какое наблюдение позволило бы выбрать между объяснениями.

13. Краткий справочник и источники

Термин	Рабочий смысл
QUIC	Защищённый транспорт поверх UDP с потоками и состоянием соединения
Stream	Упорядоченная последовательность байтов внутри соединения
HOL	Head-of-Line blocking: ожидание недоставленного начала последовательности
RTT	Round-Trip Time: время туда-обратно
ACK	Acknowledgment: подтверждение пакетов, не результата операции
PTO	Probe Timeout: тайм-аут, после которого отправляют пробы
Flow control	Ограничение новых данных ресурсами получателя
Congestion control	Ограничение отправки по состоянию пути
cwnd	Congestion window: окно перегрузки
CID	Connection ID: идентификатор для поиска состояния соединения
PSK	Pre-Shared Key: предварительно согласованный секрет
ALPN	Application-Layer Protocol Negotiation: выбор прикладного протокола
QPACK	Представление полей HTTP/3 с использованием таблиц и литералов
Alt-Svc	Alternative Services: объявление альтернативного HTTP-сервиса
DoQ	DNS over QUIC: DNS непосредственно через QUIC
DoH / DoT	DNS через HTTPS / TLS
ECH	Encrypted Client Hello: защита внутреннего ClientHello
MASQUE	Расширения HTTP для проксирования UDP и туннелирования IP
DATAGRAM	Согласуемое расширение для сообщений без восстановления содержимого

Опорные спецификации: QUIC v1, RFC 9000, TLS для QUIC, RFC 9001, recovery, RFC 9002, TLS 1.3, RFC 8446, HTTP early data, RFC 8470, HTTP/3, RFC 9114, QPACK, RFC 9204, Alt-Svc, RFC 7838, HTTPS records, RFC 9460, DoQ, RFC 9250, DATAGRAM, RFC 9221.

Нормативные свойства следует отличать от частной реализации и учебного примера. Если наблюдение не согласуется с простой моделью, полезный следующий шаг — уточнить фазу, ограничения и точку наблюдения, а не объявлять протокол сломанным.